

SMART INDIA HACKATHON 2026 · PROBLEM STATEMENT 26157



DRISHTI SAT-SA

Complete beginner documentation

Supervisory Analytics Tool for SOC Assessment — an offline examiner console for NCIIPC. This book explains the problem, the product, every major file of code, the technology choices, how to run it on a laptop, how to put it on GitHub, and how to deploy it. No prior cybersecurity or web-development experience is assumed.

Item	Value
Product	DRISHTI — SAT-SA
Team	CYBERNOVA2026
Problem	PS 26157 · NCIIPC / NTRO
Theme	Blockchain & Cybersecurity · Software
Demo scale (honest)	12 CSEs · 3,378 alerts · 235 findings · Top 100
Trust	SHA3-256 live · ML-DSA-65 next (slot, not shipping)
Hardware	4 vCPU · 16 GB RAM · no GPU
One sentence	The tool says WORTH CHECKING, never GUILTY.

Honesty box. This prototype is not a production NCIIPC system, not a SIEM, and not live monitoring. $F1 > 0.85$ is a design target on a larger extract, not a measured result of this demo. Do not quote ₹500 Cr, 1.4 billion citizens, or global SIEM TAM as DRISHTI evidence.

Contents

1. Who this document is for
2. A tiny glossary (read this first)
3. The problem in plain English
4. What DRISHTI is — and is not
5. A day in the life of an examiner
6. Technology stack and why each piece is used
7. How the pieces fit (architecture)
8. Folder map — every file
9. The data (CSV files a beginner can open in Excel)
10. Code walkthrough
11. The website pages (what each button does)
12. Analytics: execution gaps, negative space, SPS, Top 100, ML
13. Trust: SHA3-256 hash chain (and ML-DSA-65 next)
14. Run the project on your laptop
15. Docker / air-gap run
16. Put the code on GitHub (from zero)
17. Deploy the website (honest options)
18. How we validate findings
19. Limits, out of scope, and common mistakes
20. Quick reference (APIs, detectors, commands)

1. Who this document is for

You may have never written a website, never used GitHub, and never sat in a Security Operations Centre. That is fine. SAT-SA is a supervisory tool: it helps a government examiner look at SOC paperwork, not a hacker-movie “live cyber war room.”

By the end you should be able to: (a) explain the problem to a judge in two minutes, (b) point at a file and say what it does, (c) start the demo, (d) push the folder to GitHub, (e) run it in Docker as an offline box.

2. A tiny glossary (read this first)

Word	Plain meaning
NCIIPC	National Critical Information Infrastructure Protection Centre. They supervise cyber resilience of important Indian entities — not run those entities’ SOC’s.
CSE	Critical Sector Entity. A bank, grid, telecom, hospital, etc. that NCIIPC assesses.
SOC	Security Operations Centre. The CSE’s own team that watches alerts (malware, phishing...).
SIEM	Security Information and Event Management. Software such as Splunk, QRadar, Microsoft Sentinel that collects live logs. DRISHTI is not a SIEM.
Alert	A row the SIEM raised: “this host looked like malware at 14:02.” We only keep metadata (id, time, severity, asset), not packet captures.
Case	A ticket: someone investigated one or more alerts and wrote notes, then closed it.
SAT-SA	Supervisory Analytics Tool for SOC Assessment — the problem statement name. DRISHTI is our solution.
Examiner / supervisor	The NCIIPC human. The product is built for them, not for the SOC analyst.
Execution gap	The paperwork says the SOC is healthy, but the tickets show sloppy work (closed too fast, copy-paste notes).
Negative space	Something that should have happened is missing (a critical server with zero alerts). Silence is a risk.
SPS	Supervisory Priority Score 0–100. High SPS = “this CSE needs attention,” not “this CSE is guilty.”
Trinity	Every finding shows Rationale · Evidence · Confidence. The engine never closes a ticket.
Air-gap / offline	No internet. No cloud ChatGPT. The box can sit on an NCIIPC intranet.
SHA3-256	A NIST hash. We chain hashes so a finding cannot be silently edited. Live in this prototype.
ML-DSA-65	NIST post-quantum signature (FIPS 204). Reserved next step. Not shipping in the demo.
API	A URL the webpage calls, e.g. /api/overview. The browser asks the Python server for JSON data.
JSON / CSV	JSON is structured text {“key”: value}. CSV is a spreadsheet you can open in Excel.
Git / GitHub	Git = version history on your laptop. GitHub = a website that stores that history so judges can download the source.
Docker	A sealed box (“container”) that runs the app the same way on any machine, even with --network=none.

3. The problem in plain English

Imagine NCIIPC must check dozens of CSEs. Each CSE already has a SOC and a SIEM. Those SOC’s produce mountains of alerts and case notes. Today an examiner samples a few tickets by hand. That sampling keeps finding real problems that never appear on the CSE’s pretty KPI dashboard: tickets closed in three minutes, identical investigation notes, a critical power-grid server that

raised zero alerts.

Hand sampling works, but it does not scale. SAT-SA’s job is: ingest the periodic extract (not a live tap), rank CSEs that need attention, put the worst ~100 items in a queue, explain why, and leave the decision to a human.

What NCIIPC asked for (the four supervisor jobs)

- Identify entities that need supervisory attention.
- Prioritise alert samples and investigations for manual review.
- Detect operational weaknesses and cyber-resilience concerns (execution gaps + negative space).
- Improve efficiency, consistency and scale — without replacing judgement.

Out of scope (do not build these)

- Do not become the CSE’s SOC.
- Do not do real-time monitoring.
- Do not act as a SIEM.
- Do not be a central SOC for many entities.
- Do not continuously collect logs or telemetry.
- Do not be a national cyber-monitoring platform.

4. What DRISHTI is — and is not

DRISHTI (Sanskrit for “sight”) is our name for the SAT-SA prototype. It is a small website plus a Python engine that runs entirely on one machine.

DRISHTI does	DRISHTI does not
Read CSVs / ZIP / JSON the CSE already exported	Tap Splunk/QRadar live
Score CSEs with SPS 0-100	Declare a CSE guilty
Queue Top 100 for a human	Auto-close tickets
Flag execution gaps and silence	Replace the SIEM
Show Rationale · Evidence · Confidence	Call a cloud LLM
Seal rows with SHA3-256	Claim ML-DSA-65 is already shipping

Demo extract (measured, not a market TAM): 12 CSEs · 3,378 alerts · 2,408 cases · 372 assets · 235 findings · Top 100 queue. Planted ground-truth recovered 38 / 40 in this extract — that is a demo check, not a production F1 score.

5. A day in the life of an examiner

1. A CSE puts six CSV files on a USB stick (or one ZIP). No raw logs, no customer names.
2. The examiner copies the ZIP into DRISHTI (Dashboard or Workflow page). The engine runs locally.
3. Dashboard shows which CSEs have high SPS. Peer bench shows who is a statistical outlier versus their sector.
4. Review Queue lists 100 items. The examiner opens one: Trinity + SHA3. They click Accept, Reject, or False Positive.

5. Negative Space shows critical assets with zero alerts. Trends shows this extract over weeks (not live SIEM).

6. Advisory prints a short “worth checking” letter. JSON / CSV / SQLite downloads carry the sealed pack out.

The engine never says the last word. That is the whole product philosophy.

6. Technology stack and why each piece is used

Beginners often ask why not WordPress or ChatGPT. The problem statement is strict: fully offline, no cloud, no SaaS, no hosted AI. Every library below can run on a laptop after one install.

Piece	What it is	Why we chose it
Python 3.11	Language of the engine	Examiner data is tables. pandas is the usual way to slice CSVs. C++ is faster but harder to audit in a student team.
FastAPI	Tiny Python web server	Serves HTML and JSON. Typed bodies. Django is a full CMS we do not need. React/npm often needs a CDN (bad for air-gap).
Uvicorn	Process that listens on port 8000	The ASGI server FastAPI expects.
pandas	Spreadsheet-in-code	Thousands of rows, not billions. Spark/Flink are SIEM-scale clusters.
NumPy	Numeric arrays	Needed by scikit-learn and SPS / Z-score maths.
scikit-learn	Classic ML on CPU	IsolationForest + TF-IDF. No GPU. No download of a giant neural net.
Isolation Forest	Anomaly detector (ICDM 2008)	n_estimators=200 as an EXTRA flag. Rules still fire first so a judge can read a finding without trusting ML.
TF-IDF + cosine	Similar notes have cosine near 1	EG-03: cosine > 0.95, same analyst, many pairs = template investigations. No ChatGPT.
HTML/CSS/JS	The pages you see	One folder. No npm build. Browser loads index.html; app.js calls /api/*.
SHA3-256	NIST FIPS 202 hash (built into Python)	LIVE. Each finding is hashed and chained. Edits break verify().
HMAC	Local keyed seal	Stand-in for ML-DSA-65 until a PQC library is installed. Same JSON fields.
python-multipart	File uploads	USB ZIP / JSON ingest.
SQLite export	One-file database download	PS allows database exports. We snapshot; we do not run a live SIEM database.
Docker	Sealed container	Same app on any NCIIPC box. docker run --network=none.

What we deliberately did not use

- React / Next.js / npm — extra build, often a CDN.
- Cloud LLMs — forbidden by PS section 5.
- Kafka / Flink / Elasticsearch — SIEM guts, not SAT-SA.
- Live Splunk/QRadar APIs — CSEs submit extracts; we do not sit inside their SOC.
- GPU PyTorch — Isolation Forest on CPU is honest.

7. How the pieces fit (architecture)

```

USB ZIP / JSON / CSVs
  |
  v
FastAPI (app/main.py)    browser talks here
  |-- data_gen.py       first boot: 12 synthetic CSEs

```

```

|-- engine.py      rules + IF + TF-IDF + SPS + Top 100
|-- trust.py       SHA3-256 chain + HMAC
v
frontend/index.html + app.js
v
Examiner Accept / Reject / FP    (engine never closes)

```

Six layers: Ingest (schema + hash) to Store (pandas) to Detect (rules first, ML extra) to Score (SPS + peer Z) to Prioritise (Top 100) to Trust (Trinity + chain) to UI.

8. Folder map — every file

```

drishti/
  app/main.py      Web server + /api/ routes + ingest
  app/engine.py    Detectors, SPS, peers, Top 100
  app/data_gen.py  Synthetic 12-CSE extract + ground_truth
  app/trust.py     sha3(), seal(), verify()
  frontend/       index.html, app.js, styles.css, logo.png
  data/sample/    Canonical CSVs after first boot
  ARCHITECTURE.md  2-page evaluation deliverable
  README.md       Setup
  Dockerfile      Container
  requirements.txt Pinned libraries

```

9. The data (CSV files you can open in Excel)

PS section 2: periodic alert metadata, cases, workflow, escalation, disposition, assets. Minimise raw logs, pcaps, and customer PII.

File	One row is	Why
alerts.csv	One SIEM alert (id, time, severity, asset, MITRE, dwell, steps, escalation)	Fast-close and volume maths
cases.csv	One ticket with notes / root cause	EG-03 templates, EG-09 short notes
assets.csv	One system + criticality + sector	Negative space: critical + 0 alerts
analysts.csv	Who worked the queue	EG-12 overload, NS-05 silent analyst
playbooks.csv	Expected SOP	What good looks like
compliance.csv	Control to capability	Governance map
entities.csv	The 12 CSEs	Peer groups by sector
ground_truth.csv	Planted gaps	Validate page — not a production F1

On first start, if alerts.csv is missing, data_gen.generate() writes these with random seed 42 so every laptop shows the same demo.

10. Code walkthrough

10.1 app/trust.py — the seal

Each finding stores a fingerprint of itself (sha3_256) and of the previous finding (chain_hash). If someone edits an old rationale, verify() fails.

```

def seal(payload, prev_hash="GENESIS"):
    body_hash = sha3(payload)
    chain = sha3(prev_hash + body_hash)
    sig = HMAC(local_key, chain) # stand-in for ML-DSA-65
    return {sha3_256, prev_hash, chain_hash, seal, alg, sealed_at}

```

Live: SHA3-256 + HMAC. Next: swap HMAC for ML-DSA-65 (FIPS 204) without changing JSON fields. Key file: data/offline_seal.key — do not publish a real key.

10.2 app/data_gen.py — fake but honest demo data

A real extract would come from CSEs. For SIH we generate 12 named CSEs with a quality knob: low-quality CSEs get more fast-closes and silent critical assets. Some rows go into ground_truth.csv. You never run this by hand; main.py calls it on boot.

10.3 app/engine.py — the brain

- STORE — tables, findings, SPS list, audit log.
- load_csvs() — reads the folder, parses timestamps.
- _add() — one finding, immediately sealed, chained.
- run_detectors() — 12 EG then 8 NS then Isolation Forest.
- score_entities() — concern rates x weights = SPS; sector trimmed median; Z-score.
- top100() — rank, cap repeats of the same rule, keep 100.

EG-01 in English: Critical AND dwell < 10 minutes AND investigation steps < 2 AND not escalated. The KPI dashboard would show a wonderful MTTR because they closed it in five minutes without investigating.

SPS weights: Detection 20%, Investigation 20%, Escalation 15%, IR 15%, SecOps 10%, Governance 10%, Discipline 5%, Resilience 5%. High SPS = more examiner time, not guilt.

10.4 app/main.py — the waiter

FastAPI: @app.get("/api/overview") means when the browser asks for that URL, run this function and return JSON. On startup: generate demo data if needed, then run_all().

URL	Returns
/	Website (index.html)
/api/health	ok, offline, loaded
/api/overview	KPIs, SPS list, detector counts
/api/queue	Top 100
/api/negspace	Silent assets + NS findings
/api/trends	Alerts by week
/api/usecases	PS examples mapped to rules
/api/method	ML hardware / offline / explain (PS \$5)
/api/validate	Planted vs recovered
/api/audit	Hash chain tail
/api/export.json .csv .sqlite	Downloads
POST /api/ingest	ZIP or JSON or named CSV
POST /api/decide	Accept / Reject / False Positive

10.5 frontend — the face

index.html is a shell: logo, sidebar, empty main. app.js fills the page when you click a button. go("queue") to render() to queue() to fetch("/api/queue") to HTML. That is a single-page app (no full reload). openFinding draws Trinity + SHA3 + Accept/Reject/FP. Those buttons POST

/api/decide. We do not close anything in the CSE SIEM. Logo CSS must use flex-shrink: 0 so the wordmark is not clipped.

11. The website pages

Sidebar	What you see	PS mapping
Dashboard	KPIs, ingest, exports, SPS table	Identify entities
Entity Radar	8 capabilities vs sector peers	Entity indicators
Peer bench	Sorted by Z ; Z>2 flagged	FR-8 peer compare
Review Queue	Top 100 + Accept/Reject/FP	Prioritise review
Negative Space	Red = silent critical/high	PS §3.B
Trends	This extract by week	FR-16, not live SIEM
PS use cases	Nine PS examples to rules	Illustrative UCs
Workflow	Steps + ZIP + JSON + ML spec	PS §5
Trust and Audit	Chain events	FR-13
Validate	Planted vs recovered	PS §8
Advisory	Printable worth-checking letter	FR-15

12. Analytics

12.1 Execution gaps

ID	In one line
EG-01	Critical closed in <10 min, <2 steps, no escalation
EG-02	Critical closed with no escalation
EG-03	TF-IDF cosine > 0.95 template notes
EG-04	Same asset+type at least 5 times, no root-cause
EG-05	Critical asset silent while looking protected
EG-06	Fast MTTR + templates + low escalation
EG-07	Critical/High with zero investigation steps
EG-08	Escalation after SLA
EG-09	Critical notes shorter than 20 characters
EG-10	Malware closed without evidence
EG-11	Dwell exceeds severity SLA
EG-12	Analyst load > 3x team median

12.2 Negative space

ID	In one line
NS-01	Silent critical asset (0 alerts)
NS-02	Missing alert categories vs sector
NS-03	Many critical alerts, zero escalations

ID	In one line
NS-04	Volume < 10% of sector median
NS-05	Analyst closed 0 cases vs team
NS-06	Missing control evidence
NS-07	Almost no night/weekend alerts
NS-08	At least 40% of critical assets silent

12.3 Isolation Forest and Top 100

Each CSE becomes numbers (MTTR, escalation rate, silent rate). Isolation Forest (200 trees) may add ML-01. Rules still show even if ML disagrees.

score = confidence x severity_weight x (1 + cap_weight) x (1 + 0.15 x |Z|)
then cap repeats of the same rule, keep 100

13. Trust: SHA3-256 live, ML-DSA-65 next

- F-00001 hashes to H1; chain = hash(GENESIS + H1).
- F-00002 hashes to H2; chain = hash(previous + H2).
- HMAC with a local key stops a stranger forging a row.
- UI shows Chain VERIFIED or BREAK.
- ML-DSA-65 (FIPS 204, Aug 2024) is the planned production swap. Not running in the demo.

14. Run the project on your laptop

- Install Python 3.10 or 3.11 from python.org. On Windows tick Add Python to PATH.
- Open a terminal in the drishti folder.

```
cd drishti
python3 -m pip install -r requirements.txt
python3 -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Windows: if python3 fails, use py -m pip ... and py -m uvicorn ...

Leave the window open. Browser: <http://127.0.0.1:8000> . First boot about 5 seconds. 0.0.0.0 means listen on all interfaces (needed in some labs). Do not publish port 8000 to the whole internet.

Symptom	Fix
No module named uvicorn	pip install -r requirements.txt
Port 8000 in use	Close the old terminal or use --port 8001
Blank page	Ctrl+F5 hard refresh
Ingest failed	ZIP must contain alerts.csv, cases.csv, assets.csv
Logo cropped	Do not flex-shrink the logo image

15. Docker / air-gap run

```
docker build -t drishti-sat-sa:offline .
docker run --network=none -p 8000:8000 drishti-sat-sa:offline
```

--network=none means the container cannot call the internet. A fully offline build also needs wheels/ pre-downloaded. For the SIH laptop demo, section 14 is enough.

16. Put the code on GitHub (from zero)

Judges want a Source Code Link. GitHub is the usual place.

16.1 One-time setup

- Create a free account at <https://github.com> and verify email.
- Install Git from <https://git-scm.com/downloads> (Windows: Git for Windows).
- Optional: GitHub Desktop from <https://desktop.github.com> if you dislike the terminal.

16.2 Do not upload secrets

Create a file named `.gitignore` in `drishti/` :

```
__pycache__/  
*.pyc  
.venv/  
data/offline_seal.key  
.DS_Store
```

Synthetic sample CSVs are OK to publish. A real CSE extract must never go on a public repo.

16.3 Commands inside drishti/

```
git init  
git add .  
git status  
git commit -m "DRISHTI SAT-SA PS 26157 initial"
```

On github.com: New repository, name e.g. `drishti-sat-sa`, do NOT tick Add a README. Then:

```
git branch -M main  
git remote add origin https://github.com/YOUR_USER/drishti-sat-sa.git  
git push -u origin main
```

Paste that GitHub URL into the SIH portal Source Code Link. `README.md` is the Readme with Setup Instructions. Later: `git add .` then `git commit -m "message"` then `git push`.

17. Deploy the website (honest options)

Deploy means someone else can open it in a browser. SAT-SA is an NCIIPC intranet box, not a public site. GitHub Pages and Netlify only host static HTML — they cannot run FastAPI. Do not use them for the product.

17.1 What PS wants

- A server NCIIPC controls, or a sealed laptop.
- No internet: Docker `--network=none`, or a venv with libraries already installed.
- USB ZIP in, browser on `http://127.0.0.1:8000`.
- 4 vCPU, 16 GB RAM, 100 GB disk, no GPU.
- No cloud API from the UI.

17.2 SIH laptop demo (recommended)

Install Python, pip, uvicorn, open the laptop browser. Record a 2-minute video: Dashboard, Queue Trinity, Negative Space, Accept, Audit SHA3, ZIP ingest. Upload Drive or unlisted YouTube as Demo Video.

17.3 Same Wi-Fi teammate

With `--host 0.0.0.0`, others on LAN can open `http://YOUR_LAN_IP:8000`. Not the public internet. Turn it off after.

17.4 Public cloud demo (not NCIIPC-compliant)

Render / Railway / a campus VM can run Docker for a synthetic-data demo URL only. Never upload real CSE data. Never call that production.

Portal item	Where it lives
Source Code Link	GitHub URL from section 16
Readme with Setup	drishti/README.md
Architecture (max 2 pages)	Print ARCHITECTURE.md to PDF
Demo video (max 2 min)	Screen recording of the local site
Technical presentation	6-slide idea PPT including title; 5-slide tech deck only if a later round asks

18. How we validate findings

- `data_gen` plants known EG/NS rows into `ground_truth.csv`.
- Validate page matches planted rule + entity (and alert/asset when present).
- This demo recovered 38 / 40 planted rows and also fires extra rules. Extra is expected.
- Design target $F1 > 0.85$ is for a larger labelled extract — not a stamp on these 12 CSEs.
- Human baseline (two examiners on High items) is future work, not a fake chart.

19. Limits, out of scope, and common mistakes

- Do not tell a judge DRISHTI is deployed at NCIIPC.
- Do not show global SIEM TAM as SAT-SA viability.
- Do not say ML-DSA-65 is live.
- Do not say $F1 > 0.85$ was measured here.
- Do not ingest pcaps or personal IDs.
- JSON ingest is a file pack with alerts/cases/assets arrays, not a live SIEM API key.
- This demo is a single supervisor console (roles in architecture are future).

20. Quick reference

One-liner: DRISHTI is an offline supervisory analytics platform built for NCIIPC (PS 26157). SHA3-256 is live. ML-DSA-65 is next.

```
python3 -m pip install -r requirements.txt
python3 -m uvicorn app.main:app --host 0.0.0.0 --port 8000

git init && git add . && git commit -m "DRISHTI SAT-SA"
git remote add origin https://github.com/USER/drishti-sat-sa.git && git push -u origin main

fastapi==0.115.6
uvicorn==0.34.0
pandas==2.2.3
numpy==2.2.2
scikit-learn==1.6.1
python-multipart==0.0.20
```

If a slide disagrees with this PDF, the PDF and the running code win. Do not invent metrics on slides.

CYBERNOVA2026 · PS 26157 · DRISHTI SAT-SA · @SIH Idea submission